Computer Vision

Feature Detection and Extraction

Objectives of this lecture

- To understand the concept of feature detection
- To know the properties of good features
- To learn how to extract features from images
- To learn the Harris corner detection algorithm
- To learn the Scale Invariant Feature Transform (SIFT)

What are features and what is their use?

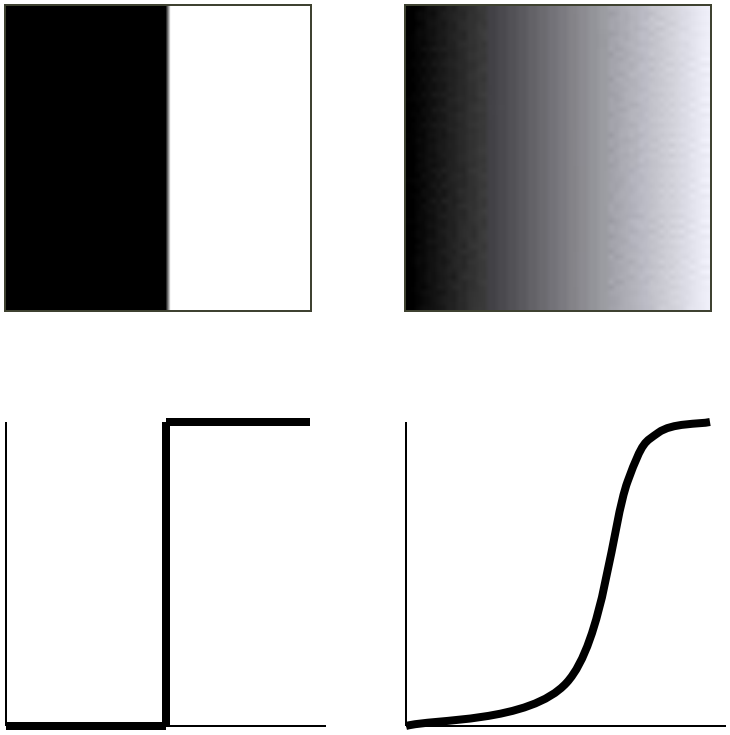
- Feature is a piece of information that describes an image or a part of it
  - Corners
  - Edges
  - Circles, ellipses, lines, blobs etc

- Features are useful to match two images
  - Find common points in two images to stitch them
  - Object recognition, face recognition
  - Structure from motion, stereo correspondence

- **Why not match pixels values?** – Pixel values change with light intensity, colour and direction. They also change with camera orientation.

What are edges in an image?

- Edges are those places in an image that correspond to object boundaries.
- Edges are pixels where image brightness changes abruptly.



Brightness vs. Spatial Coordinates

More About Edges

- An edge is a property attached to an individual pixel and is calculated from the image function behavior in a neighborhood of the pixel.
- It is a **vector variable** (**magnitude** of the gradient, **direction** of an edge**)** .

Image To Edge Map

Edge Detection

- Edge information in an image is found by looking at the relationship a pixel has with its neighborhoods.
- If a pixel's gray-level value is similar to those around it, there is probably not an edge at that point.
- If a pixel's has neighbors with widely varying gray levels, it may present an edge point.

Edge Detection Methods

- Many are implemented with convolution mask and based on discrete approximations to differential operators.
- Differential operations measure the rate of change in the image brightness function.
- Some operators return orientation information. Other only return information about the existence of an edge at each point.

Derivatives

- Edges are locations with high image gradient or derivative
- Estimate derivative using finite difference

$$\frac{\partial}{\partial x} I(x_0, y_0) \approx I(x_0 + 1, y_0) - I(x_0, y_0)$$

- Problem?

Roberts Operator

- ↘ Mark edge point only
- ↘ No information about edge orientation
- ↘ Work best with binary images
- ↘ Primary disadvantage:
  - High sensitivity to noise
  - Few pixels are used to approximate the gradient

Roberts Operator (Cont.)

- First form of Roberts Operator

$$\sqrt{\left[I(r,c)-I(r-1,c-1)\right]^2+\left[I(r,c-1)-I(r-1,c)\right]^2}$$

- Second form of Roberts Operator

$$|I(r,c)-I(r-1,c-1)|+|I(r,c-1)-I(r-1,c)|$$

$$h_1=\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix} \qquad h_2=\begin{bmatrix}0 & 1\\ -1 & 0\end{bmatrix}$$

Smoothing

- Reduce image noise by smoothing with a Gaussian

$$J = G * I$$

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

Prewitt Operator

- Avoiding noise with box-filter mask

$$y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \qquad x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Edge Magnitude = $\sqrt{x^2 + y^2}$ Edge Direction = $\tan^{-1}\left[\frac{y}{x}\right]$

Sobel Operator

- Looks for edges in both horizontal and vertical directions, then combine the information into a single metric.
- The masks are as follows:

$$y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \qquad x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Edge Magnitude = $\sqrt{x^2 + y^2}$ Edge Direction = $\tan^{-1}\left[\frac{y}{x}\right]$

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & +1 \end{bmatrix} \qquad \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

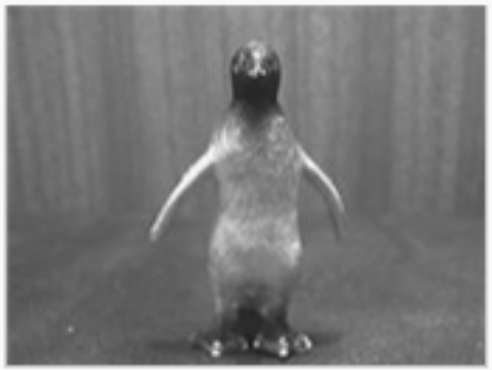
and the two derivatives $\mathbf{G_x}$ can th$\mathbf{G_y}$ re be computed as

$$\mathbf{G_x} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & +1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G_y} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} * \mathbf{A}$$

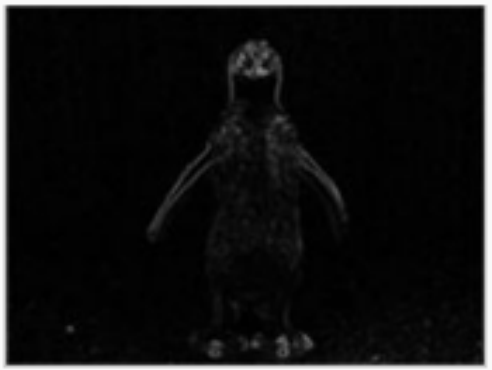
In certain implementations, this separable computation may be advantageous since it implies fewer arithmetic computations for each image point.

Example

Because the result of the Sobel operator is a 2-dimensional map of the gradient at each point, it can be processed and viewed as though it is itself an image, with the areas of high gradient (the likely edges) visible as white lines. The following images illustrate this, by showing the computation of the Sobel operator on a simple image.



Grayscale image of plastic figure of a penguin



Normalised sobel gradient image of penguin figure

Properties of Good Features

- Repeatability
  - Should be detectable at the same locations in different images despite changes in viewpoint and illumination

- Saliency (descriptiveness)
  - Same points in different images should have similar features
  - And vice-versa

- Compactness (affects speed of matching)
  - Fewer features
  - Smaller features

Feature Extraction has Two Steps

1. Feature detection
    - Where to extract features
    - Not all locations are good to extract features

2. Feature extraction
    - Orientation of the extraction window
    - What to encode in the feature

Corners are Simple Features

- Can you see common feature point between the two images?

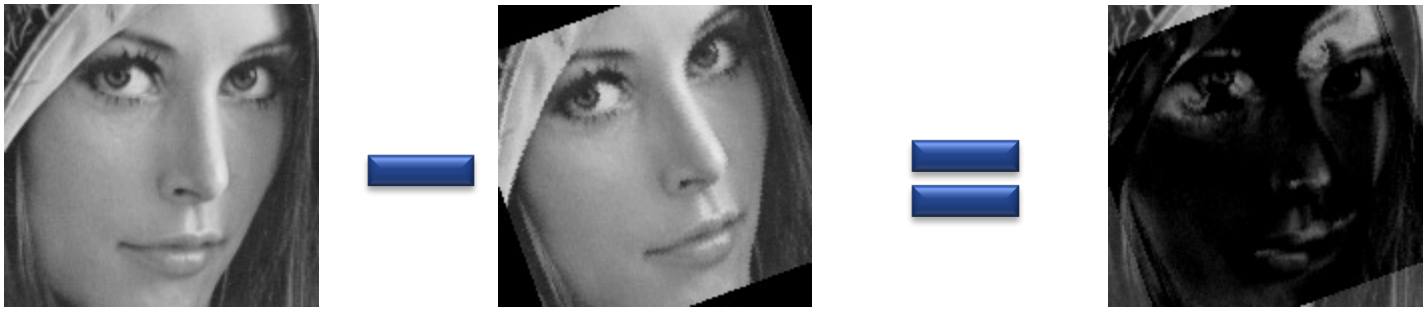
Applications of Feature Matching

- Image alignment and stitching
- 3D structure from motion
- Motion tracking
- Robot navigation
- Image retrieval
- Object and face recognition
- Human action recognition

Image Stitching (to generate panorama)

Image Alignment

- Without alignment, images of the same person or object will not match



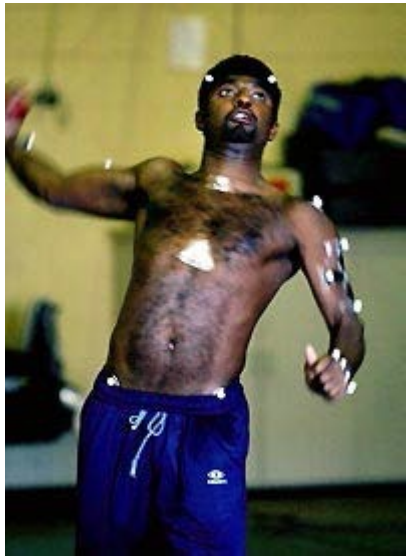
RMSE > 5000

Structure from Motion (SfM)

- Building Rome in a day
- Matching feature points in unordered image collections
- Software by Noah Snavely, Cornell University

Motion Tracking

- Is this bowler doing an underarm?
- Track his joint positioins
- Use artificial markers to make detection and tracking easy



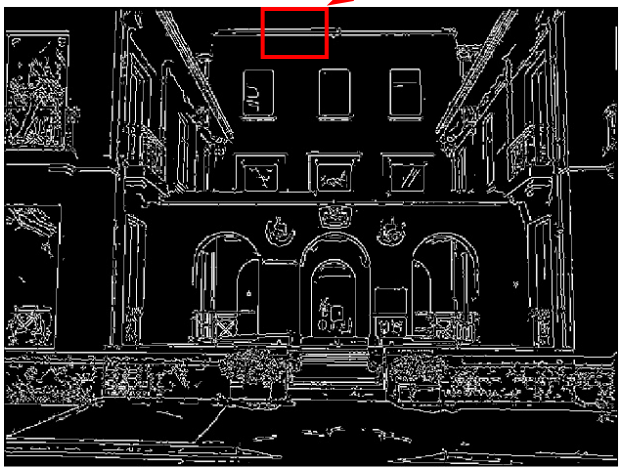
- Can be done without markers
- If we can detect the same points in all video frames

Feature Detectors – Classic and State of the Art

Feature	Detection	Extraction	OpenCV	Published
Harris	Yes	No	Yes	1988
KLT	Yes	No	Yes	1994
LBP	No	Yes	Yes	1994
SIFT	Yes	Yes	Yes	IJCV 2004
FAST	Yes	No	Yes	ECCV 2006
SURF	Yes	Yes	Yes	CVIU 2008
BRIEF	No	Yes	~	ECCV 2010
ORB	Yes	Yes	Yes	ICCV 2011
BRISK	Yes	Yes	Yes	ICCV 2011
FREAK	Yes	Yes	Yes	CVPR 2012

Canny Edge Detection

Original image



Strong + connected weak edges

Strong edges only

Weak edges

courtesy of G. Loy

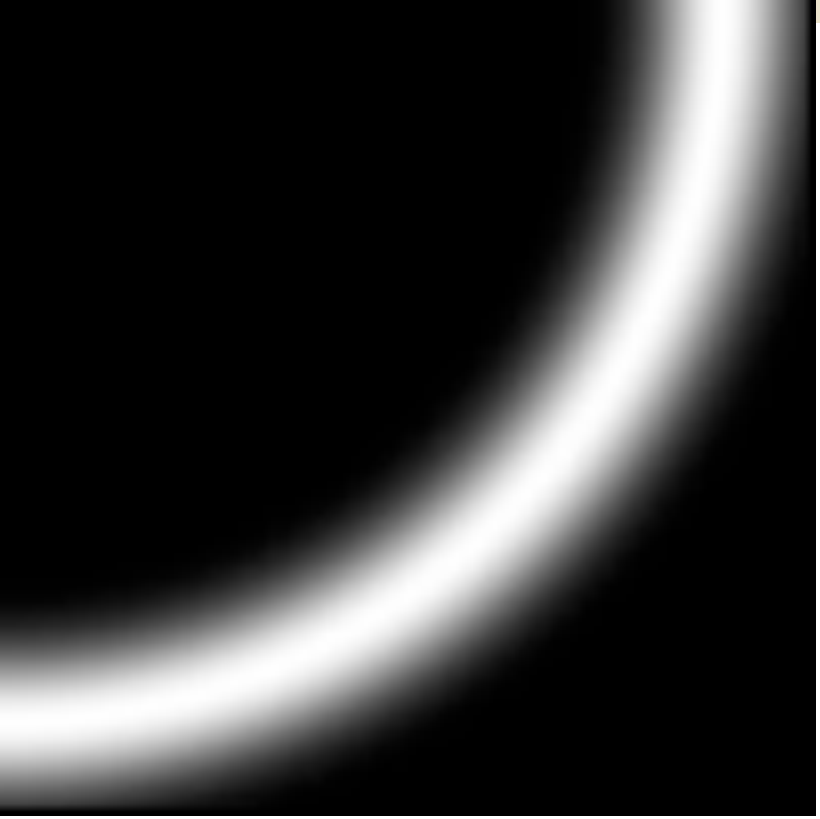
Edge Hysteresis

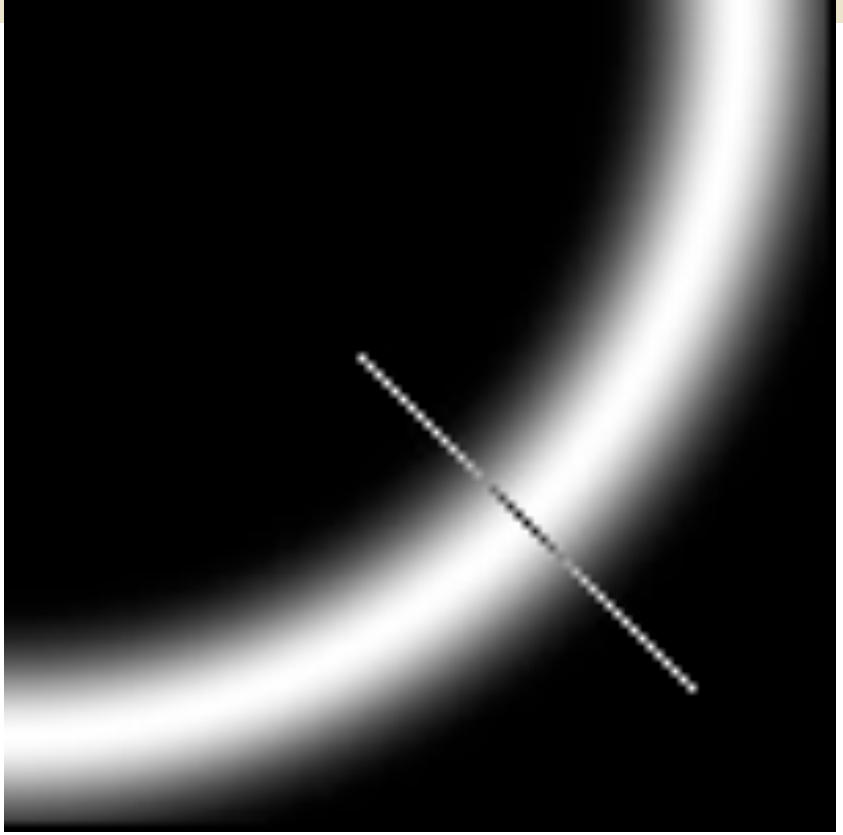
- **Hysteresis**: A lag or momentum factor
- Idea: Maintain two thresholds k_{high} and k_{low}
  - Use k_{high} to find strong edges to start edge chain
  - Use k_{low} to find weak edges which continue edge chain
- Typical ratio of thresholds is roughly

$$k_{high} / k_{low} = 2$$

Canny Edge Detection

- Compute edge strength and orientation at all pixels
- “Non-max suppression”
  - Reduce thick edge strength responses around true edges
- Link and threshold using “hysteresis”
  - Simple method of “contour completion”

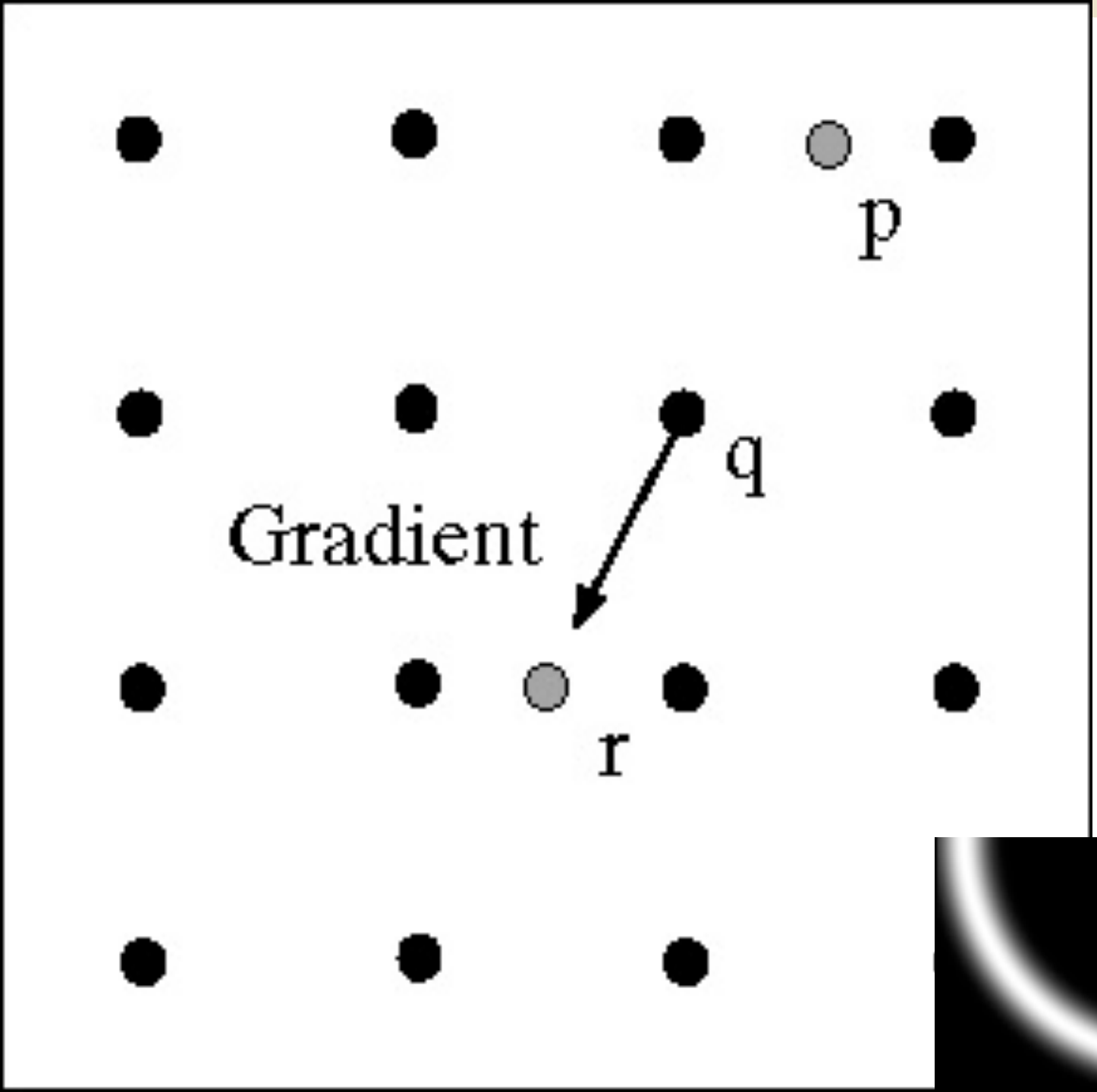




Non-maximum suppression:

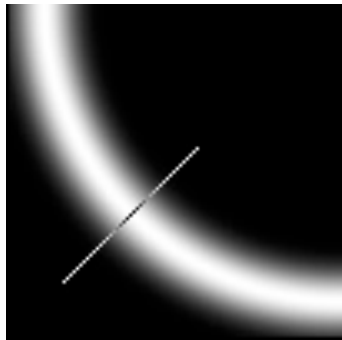
Select the single maximum point across the width of an edge.

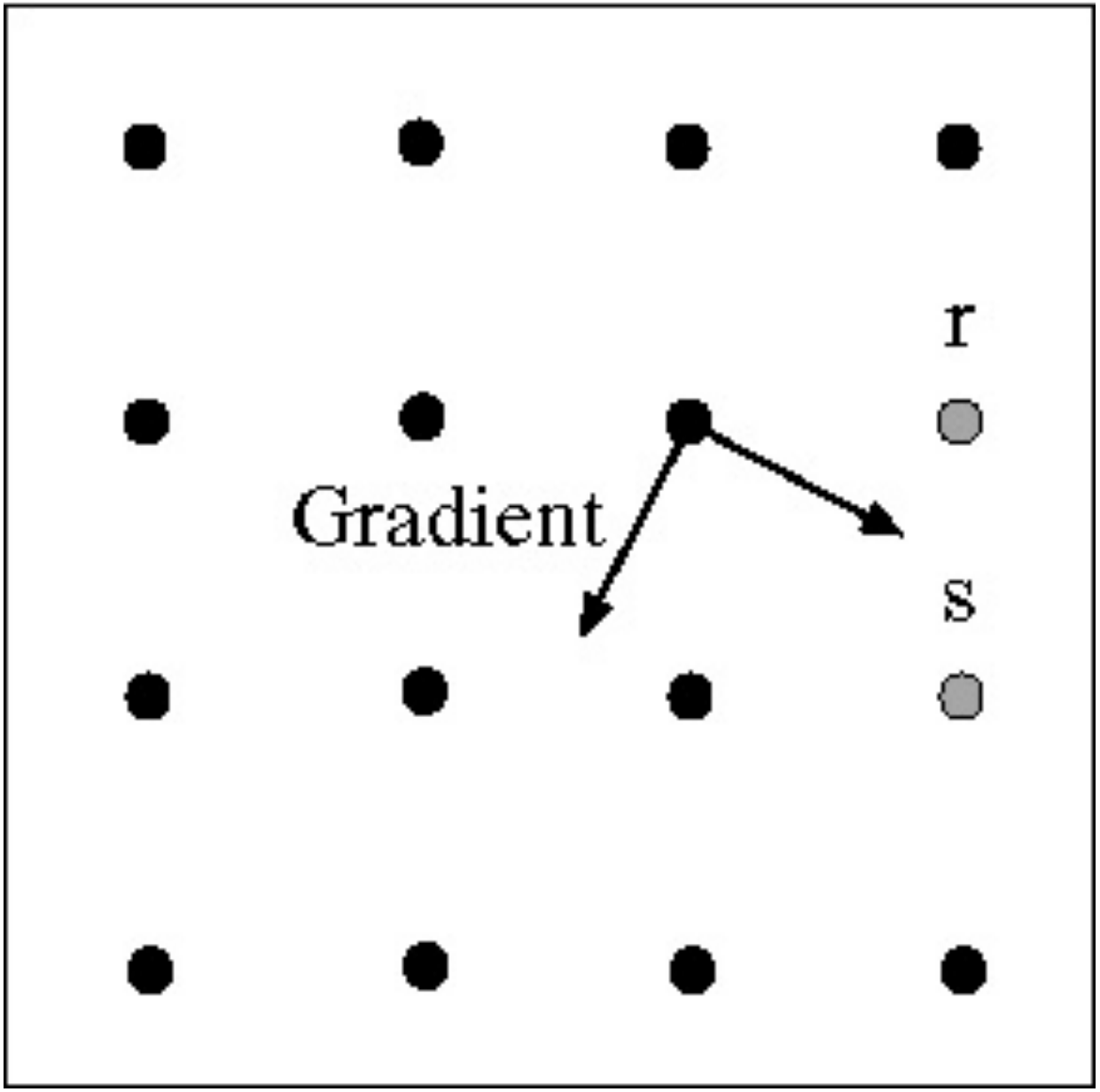
Slides by D. Lowe



Non-maximum suppression

At q, the value must be larger than values interpolated at p or r.





Linking to the next edge point

Assume the marked point is an edge point.

Take the normal to the gradient at that point and use this to predict continuation points (either r or s).

Examples:
Non-Maximum Suppression

Original image

Gradient magnitude

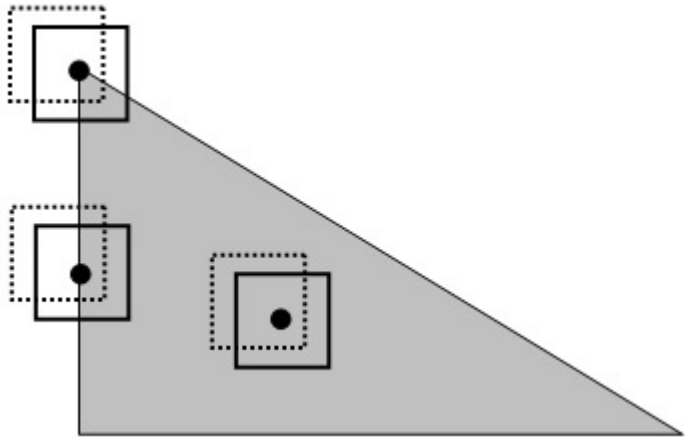
courtesy of G. Loy

Non-maxima suppressed

Slide credit: Christopher Rasmussen

Corner Detection: Basic Idea

- Where two edges meet
- Where X and Y gradients are both high??

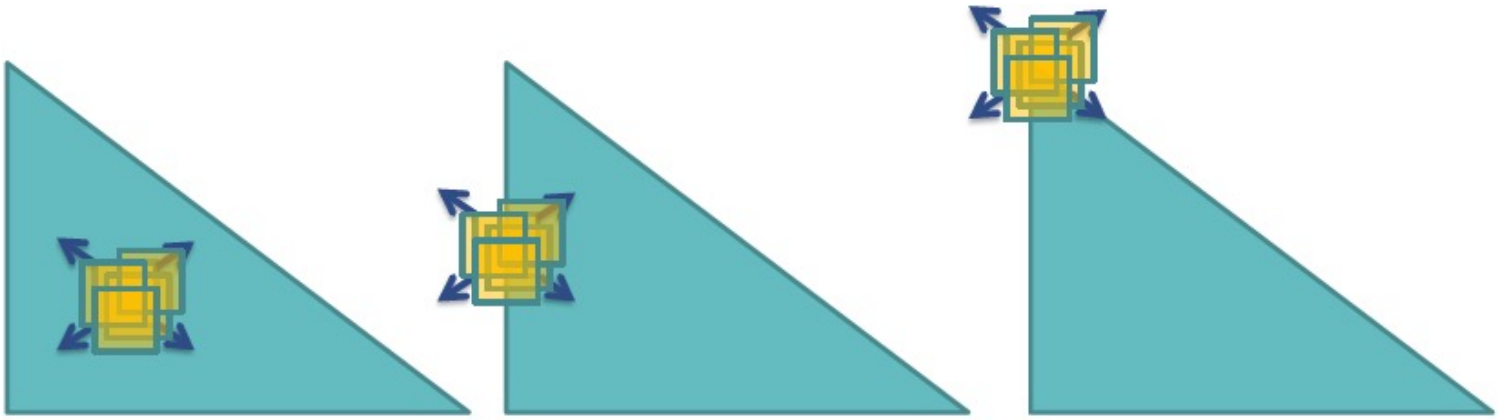


Harris Corner Detector

- A corner is a point around which the gradient has two or more dominant directions

- Corners can be repeatably detected under varying illumination and view point changes

Harris Corner Detector: Basic Idea



Flat region:
No change in
all directions

Edge:
No change along
the edge direction

Corner:
Change in
all directions

Harris corner detector mathematically determines these three cases

Harris Corner Detector : Mathematics

↘ Change in appearance of window $w(x, y)$ for the shift $[u, v]$:

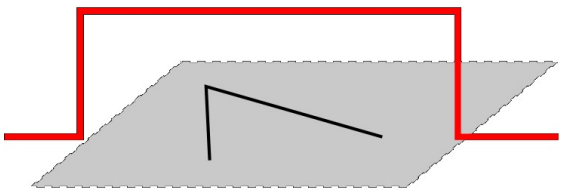
$$E(u, v) = \sum_{x,y} w(x, y)[I(x + u, y + v) - I(x, y)]^2$$

Window function

Shifted intensity

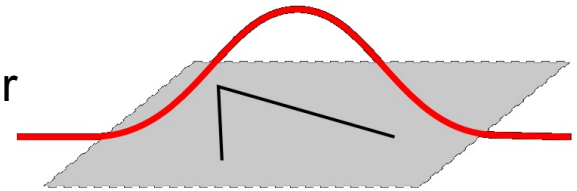
Intensity

Window function $w(x, y) =$



Box i.e. 1 inside, 0 outside

or



Gaussian

Harris Corner Detector : Intuition

- Change in appearance of window $w(x, y)$ for the shift $[u, v]$:

$$E(u, v) = \sum_{x,y} w(x, y)[I(x + u, y + v) - I(x, y)]^2$$

For uniform intensity patches, this term will be zero. Recall that we want to find patches where the variation is large. Hence, we want to find patches where $E(u, v)$ is LARGE.

Harris Corner Detector : Derivation

$$I(x+u, y+v) \approx I(x,y) + uI_x(x,y) + vI_y(x,y)$$

First order Taylor series approximation

$$[I(x+u, y+v) - I(x,y)]^2$$

Recall the square term of $E(u,v)$

$$\approx [I(x,y) + uI_x(x,y) + vI_y(x,y) - I(x,y)]^2$$

$$= [uI_x(x,y) + vI_y(x,y)]^2 = [uI_x + vI_y]^2$$

$$= u^2 I_x^2 - v^2 I_y^2 + 2uvI_xI_y$$

Removing (x,y) for readability

$$= \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} I_x^2 & I_xI_y \\ I_xI_y & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Harris Corner Detector : Mathematics

- Window function can be simply w = 1
- Product of first derivatives of the image

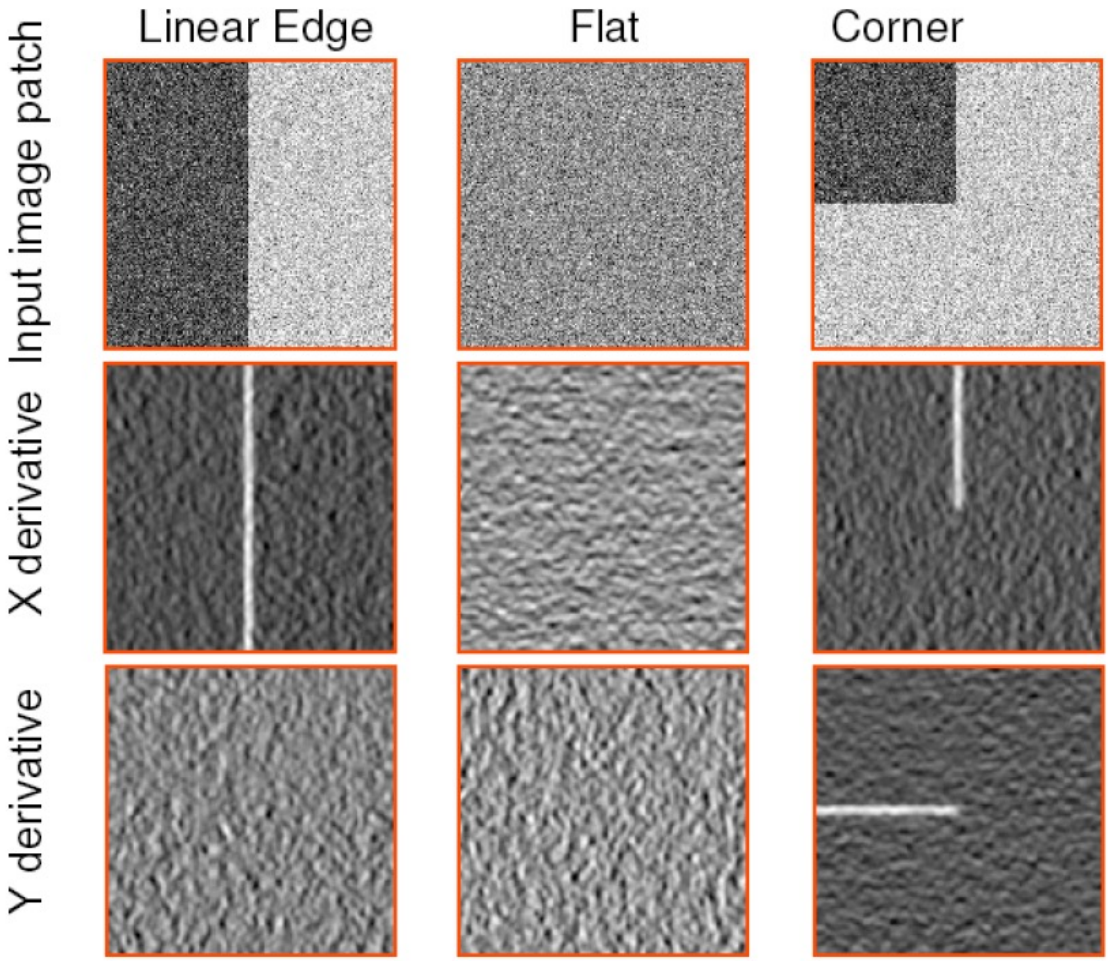
$$M = \sum w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$M = \sum w(x,y) \begin{bmatrix} I_x \\ I_y \end{bmatrix} \begin{bmatrix} I_x & I_y \end{bmatrix}$$

$$M = \sum w(x,y) \Delta I (\Delta I)^T$$

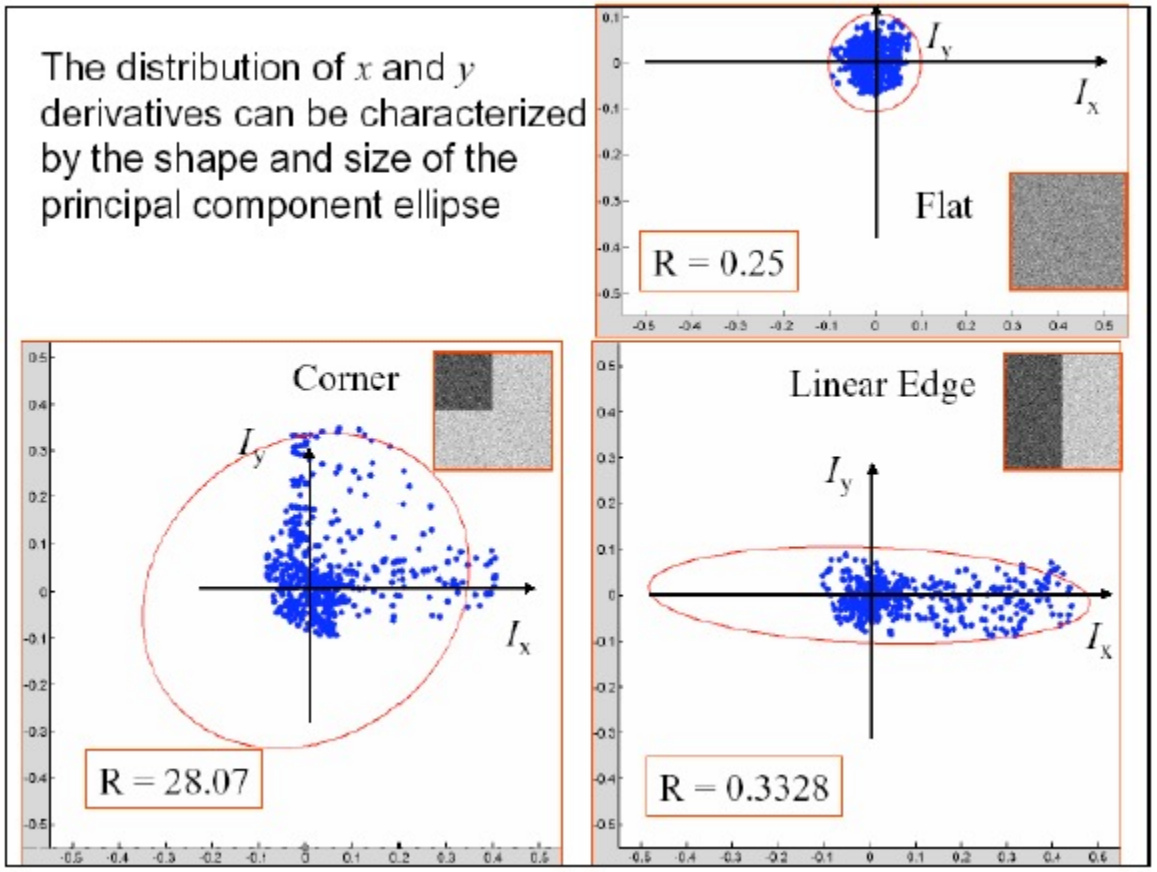
$$E(u,v) = \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

Harris Corner Detector : Intuitive Example

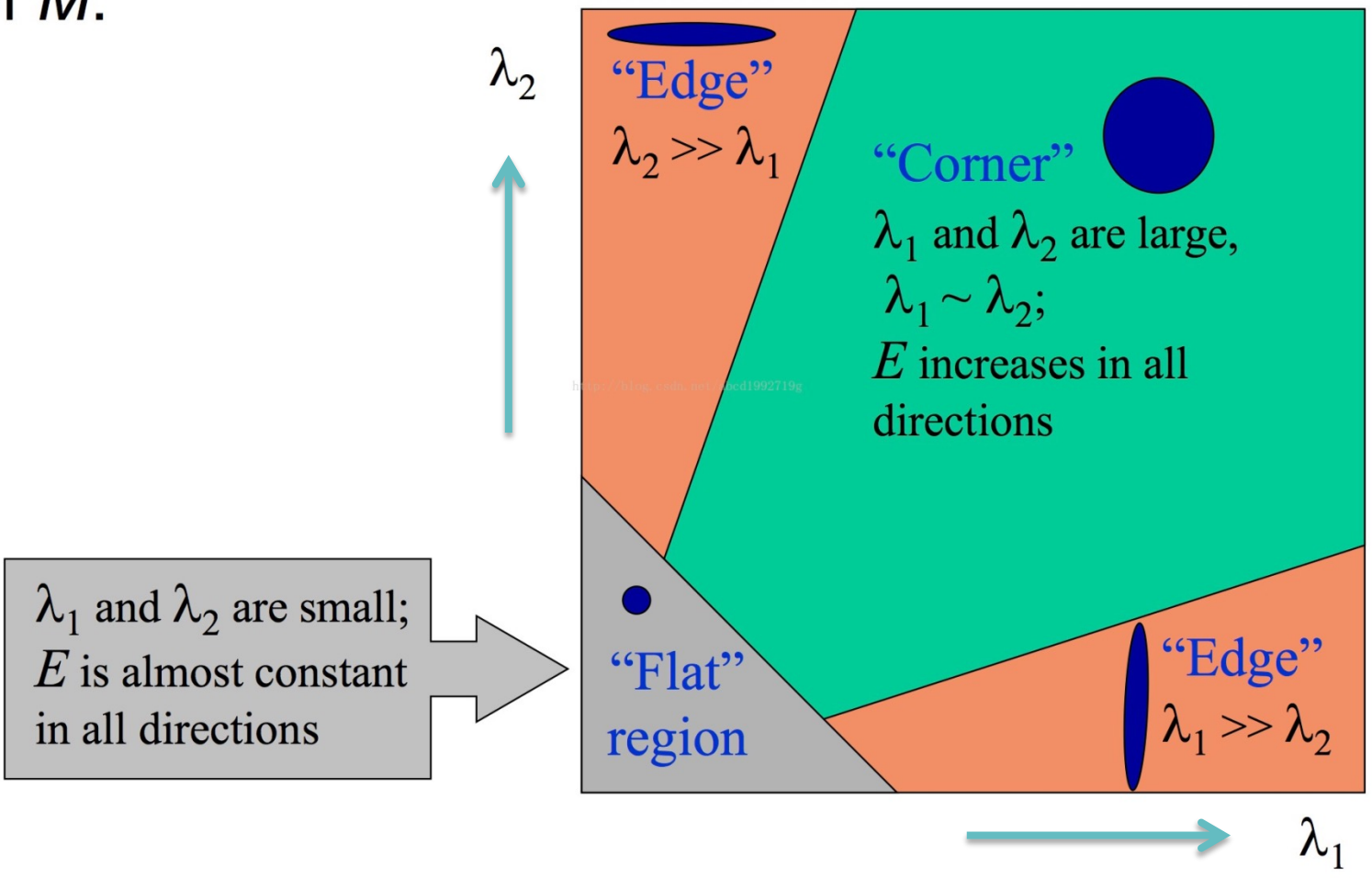


Treat gradient vectors as a set of (dx, dy) points with center of mass at (0,0)

Plotting Derivatives as 2D Points for PCA



Classification of image points using eigenvalues of *M*:



Harris Corner Detector: Cornerness Measure

Measure of corner response:

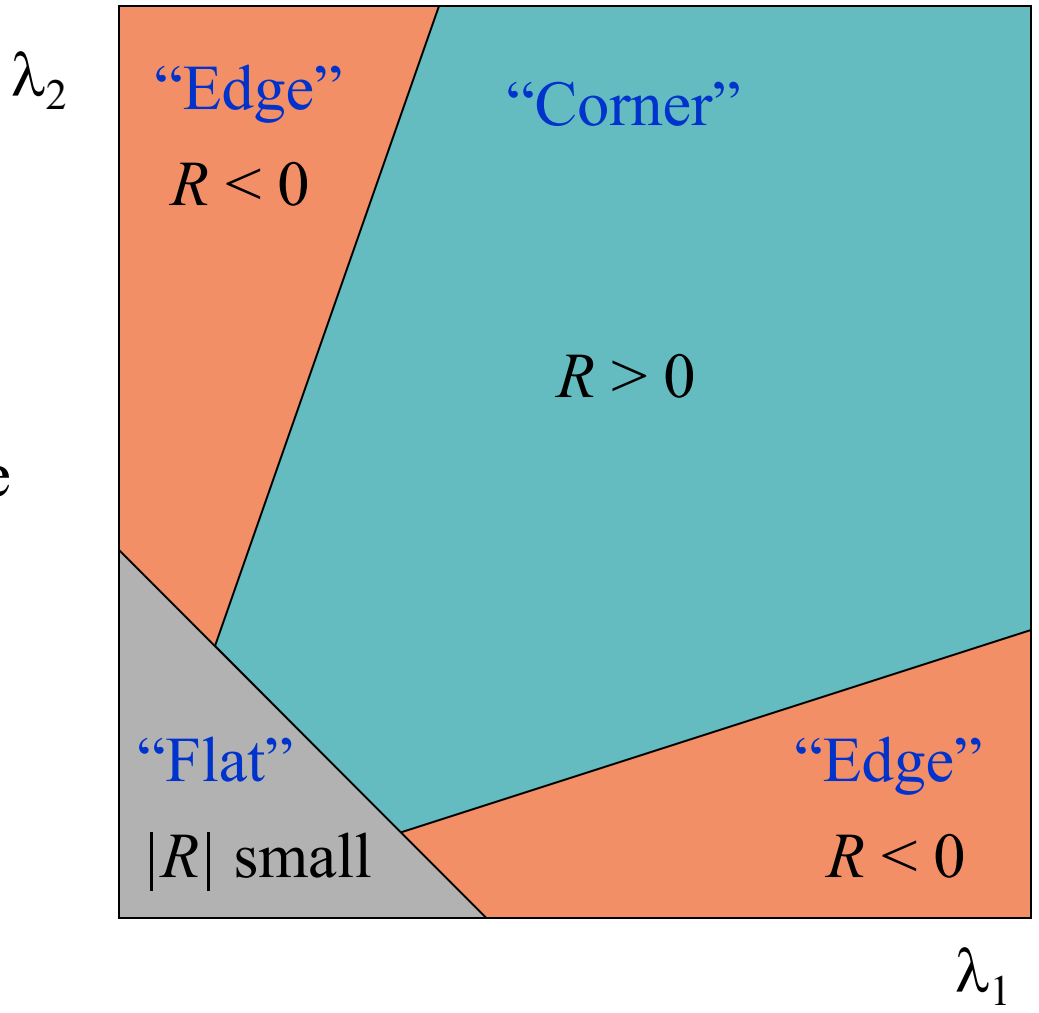
$$R = \det M - k\left(\operatorname{trace} M\right)^2$$

$$\det M = \lambda_1 \lambda_2$$
$$\operatorname{trace} M = \lambda_1 + \lambda_2$$

(k – empirical constant, k = 0.04-0.06)

Harris Corner Detector: Corner Response

- R depends only on eigenvalues of M
- R is large for a corner
- R is negative with large magnitude for an edge
- $|R|$ is small for a flat region



Harris Corner Detection in Matlab

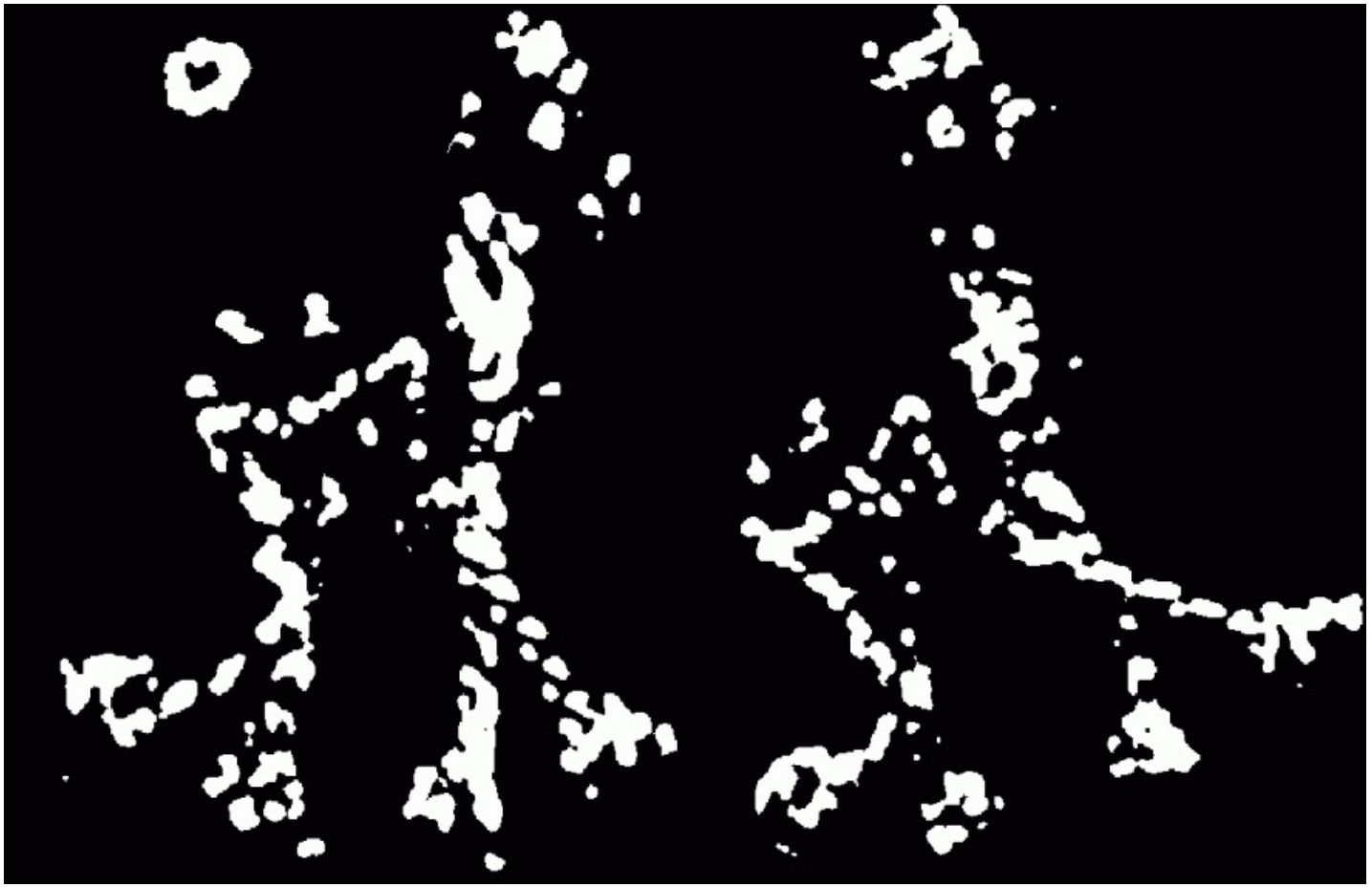
- C = corner(img); % returns the locations of Harris corners in img
- Type help corner to see details of the function
- You can try different sensitivity levels and filter values.

Algorithm : Harris Corner Detector

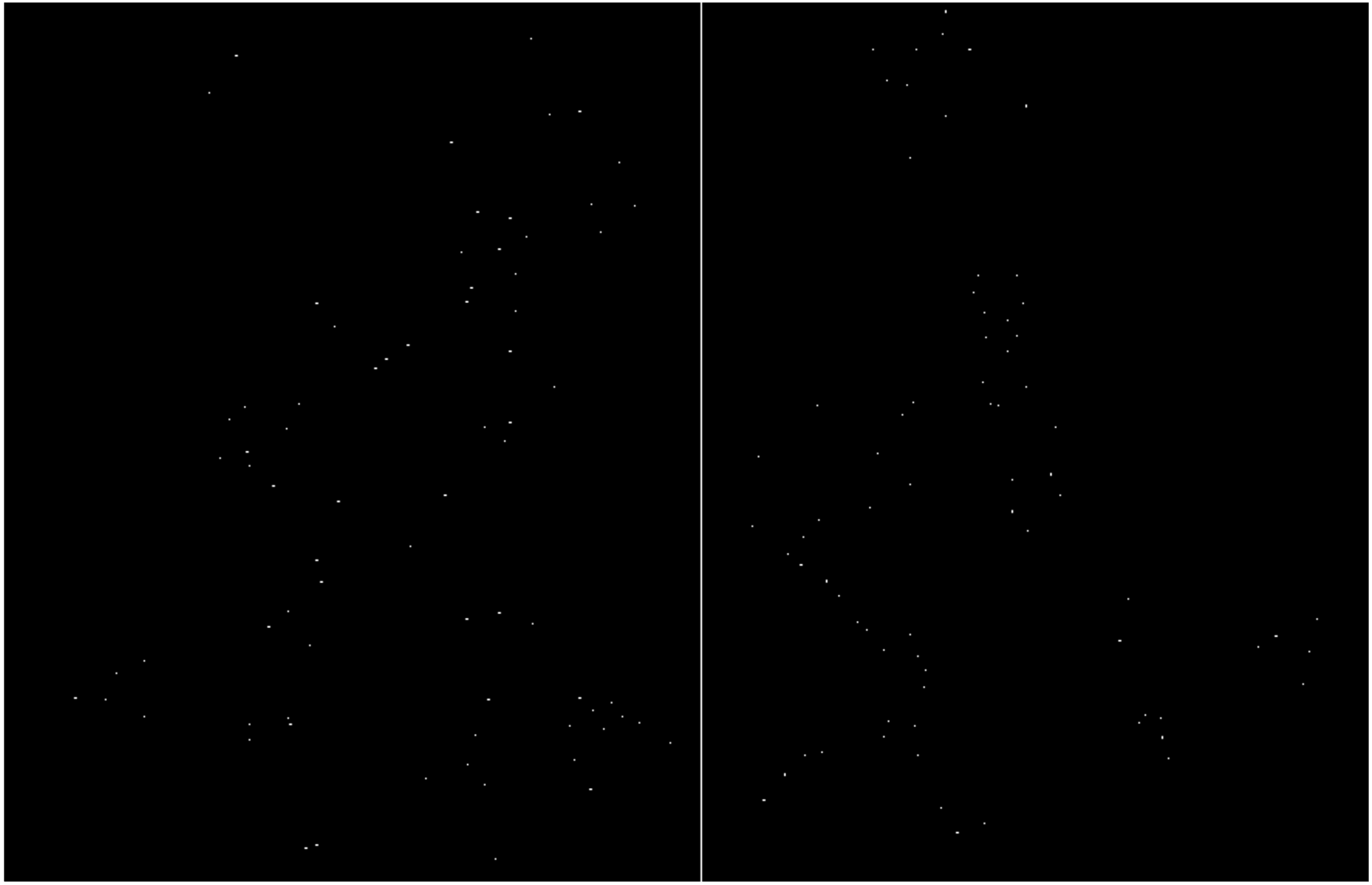
1. Computer x and y derivatives I_x and I_y of the input image
2. Computer products of derivatives $I_x I_x$, $I_x I_y$ and $I_y I_y$
3. For each pixel, compute the matrix M in a local neighborhood
4. Compute the corner response R at each pixel
5. Threshold the value of R to select corners
6. Perform non-maximum suppression

Harris Corner Detector : Example

Points with R > threshold



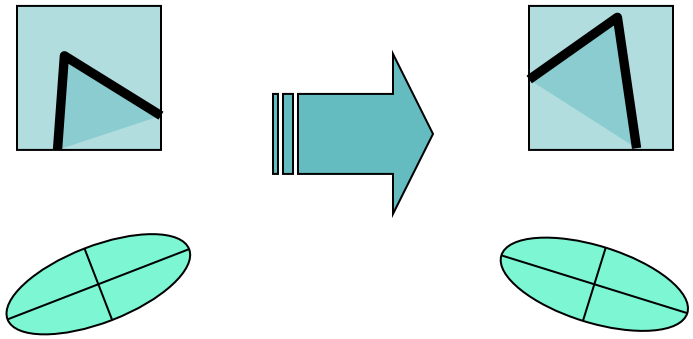
Take only the points of local maxima of R



Harris Corners

Properties of Harris Corners

- Rotation Invariance



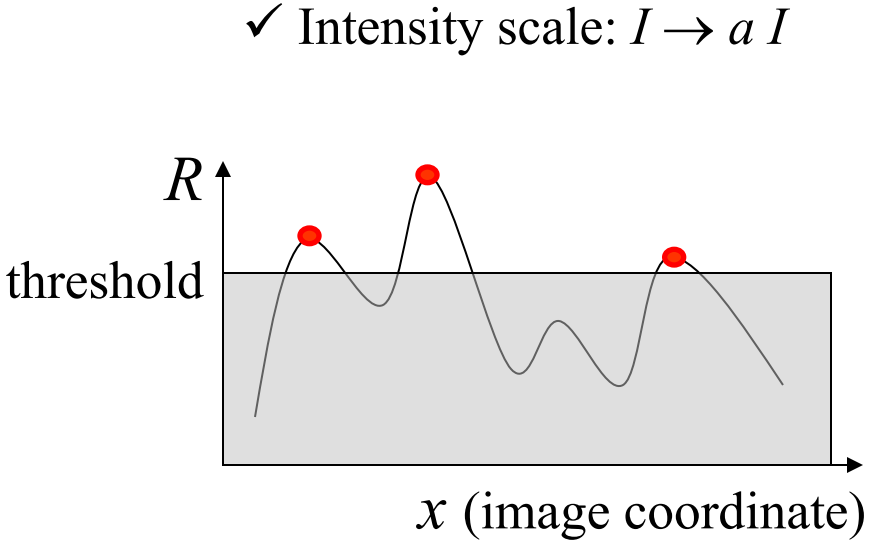
- Ellipse rotates but the shape (i.e. eigenvalues) remain the same
- Corner response R is invariant to image rotation.

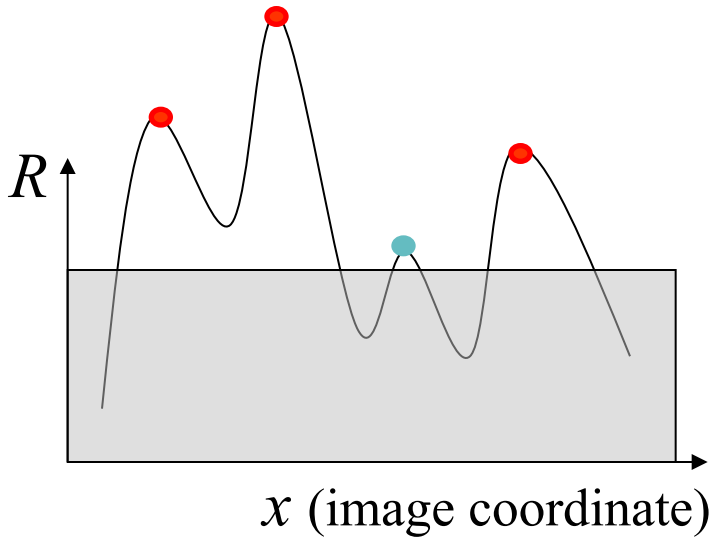
Properties of Harris Corners

↘ Partial invariance to affine intensity change

✓ Only derivatives are used => invariance to intensity shift $I \rightarrow I + b$

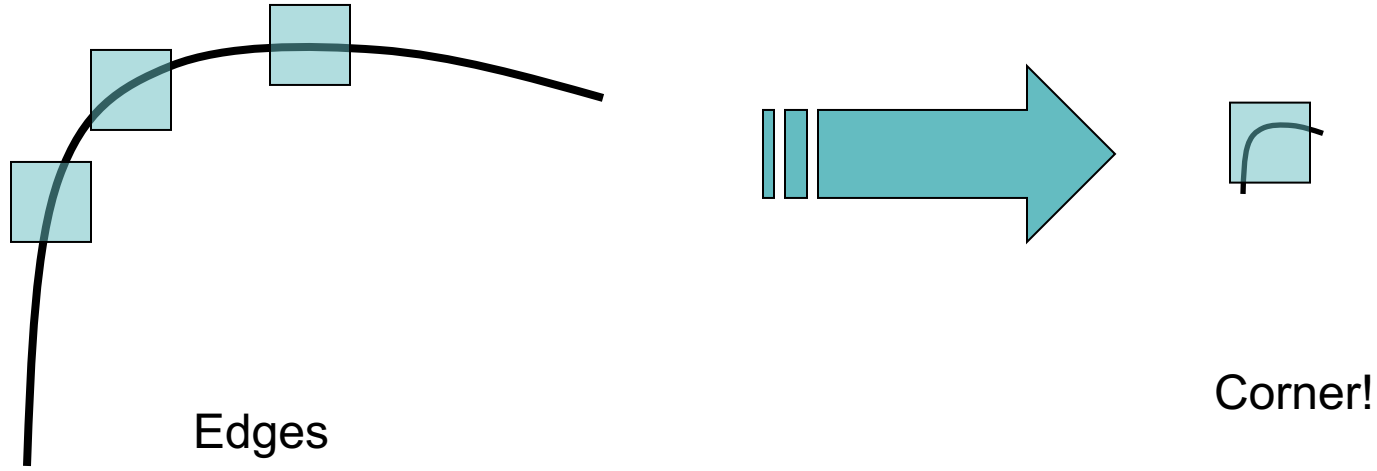
✓ Intensity scale: $I \rightarrow a\, I$





Properties of Harris Corners

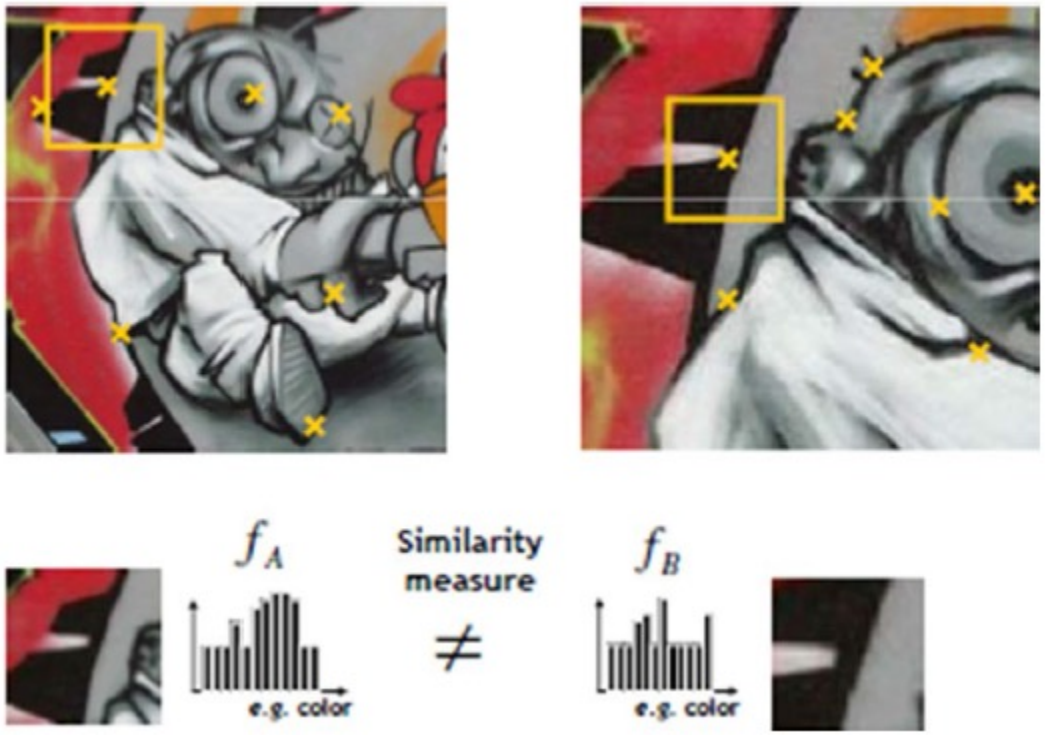
- NOT invariant to image scale
- Corner at one scale may not be a corner at another
- Scale is user specified parameter



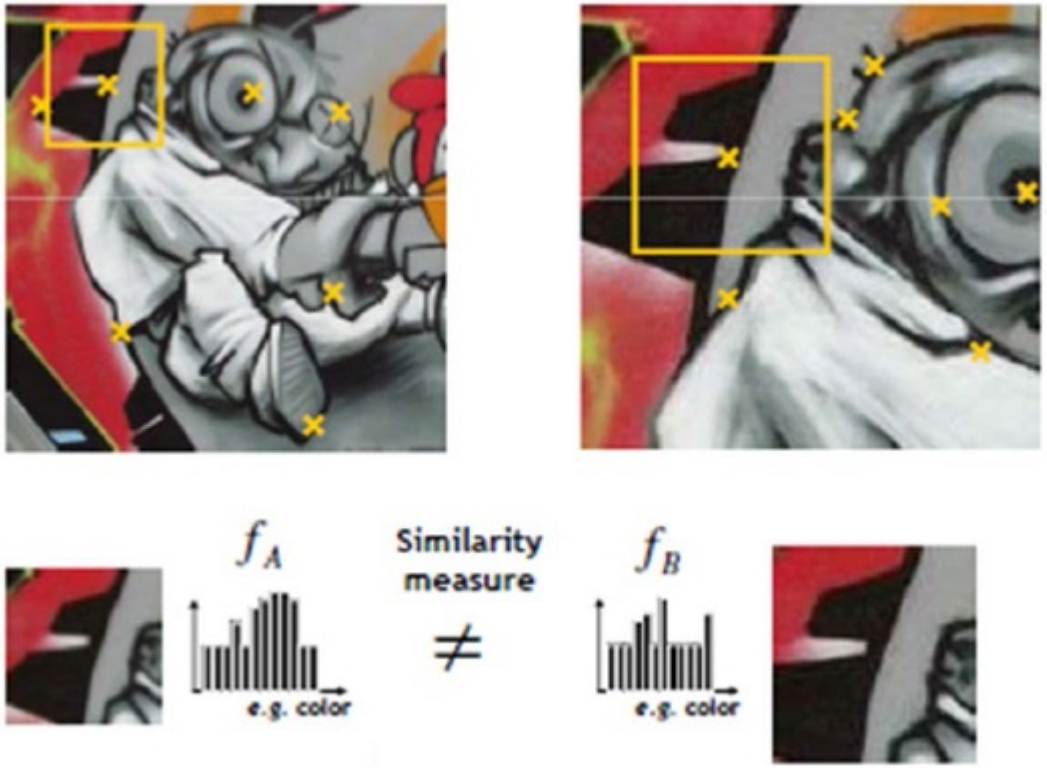
What Next?

- We have found interest points, how do we extract features?
- Take a square region around the corner point. But how big?
- What if the images to be matched have different scales?
- What if the object is rotated in the second image?

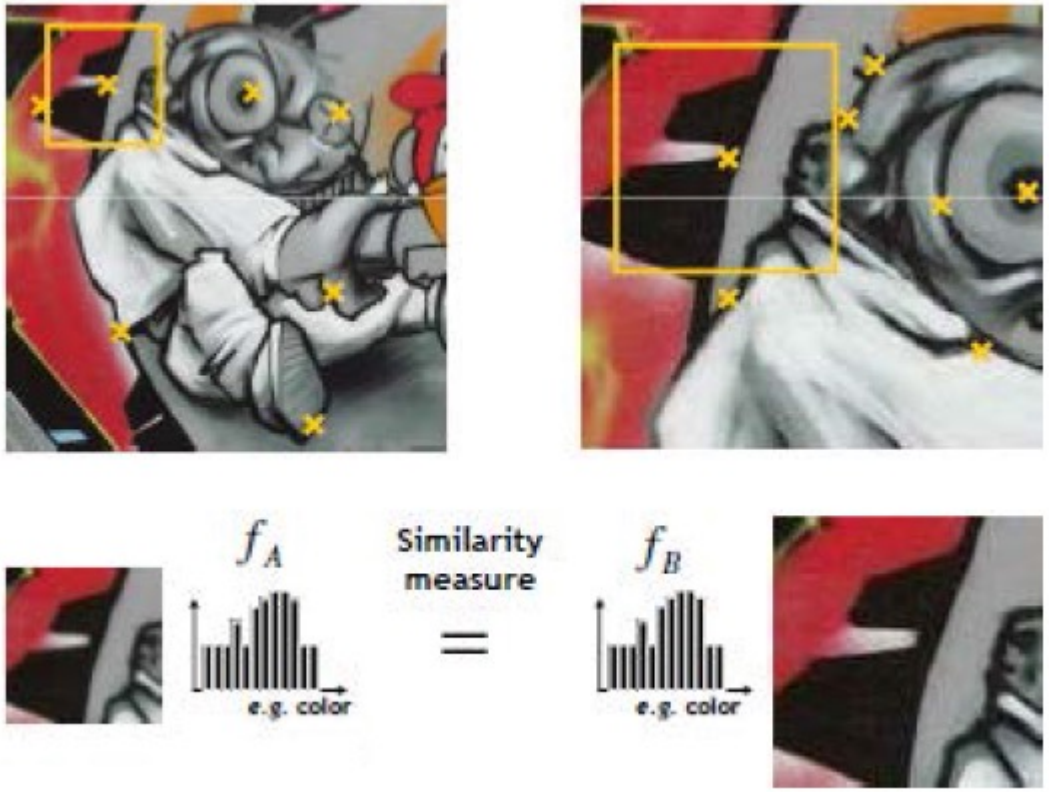
Scale Different



Exhaustive Search



Try Different Window Sizes



Can we automatically find the scale of an interest point?

- What would be the scale for Harris corner?
- Can we efficiently find interest points and their scales together?
- Such points are said to be scale invariant.
- The search for these points is referred to as scale space search.

Laplacian of Gaussian (LoG) blob detector

Laplacian is the sum of 2nd derivatives in all (Cartesian) dimensions. For an image

$$\nabla^2 I(x,y) = \frac{\partial^2 I(x,y)}{\partial x^2} + \frac{\partial^2 I(x,y)}{\partial y^2}$$

Since derivatives are very sensitive to noise, images are first smoothed with a Gaussian filter. This two step operation is called Laplacian of Gaussian (LoG)

Laplacian and Gaussian are linear filter and can be combined

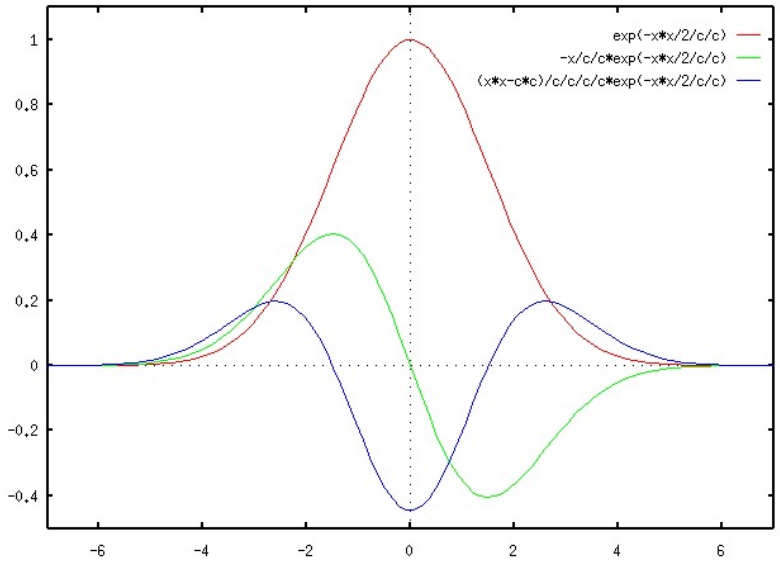
$$\nabla^2[f(x,y) \otimes G(x,y)] = \nabla^2 G(x,y) \otimes f(x,y)$$

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

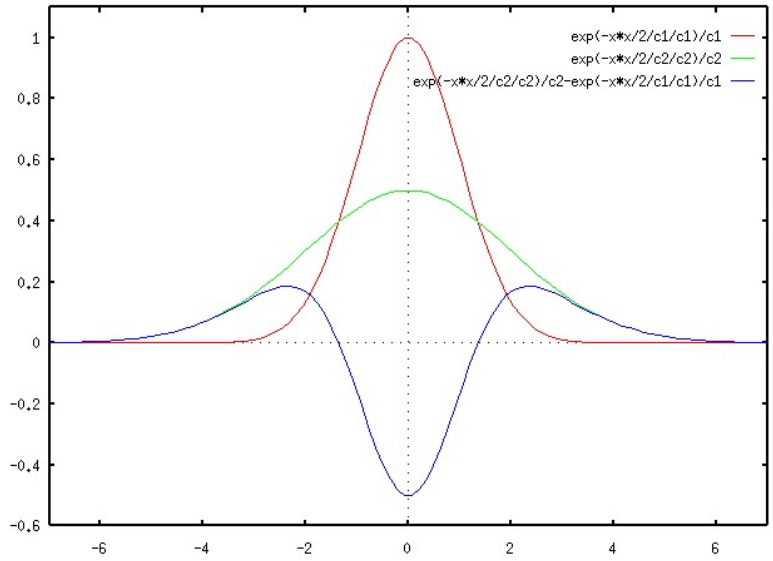
$$\mathrm{LoG}(x,y) = \frac{1}{\pi\sigma^4}\left[\frac{x^2+y^2}{2\sigma^2} - 1\right] e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

Laplacian of Gaussian : 1-D case

- Notice the similarity with edge detection
- Notice the similarity with Difference of Gaussian (DoG)

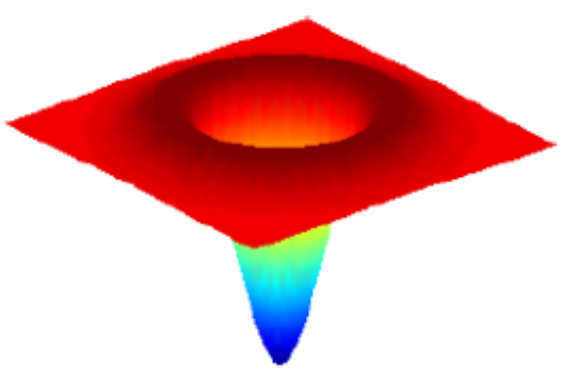


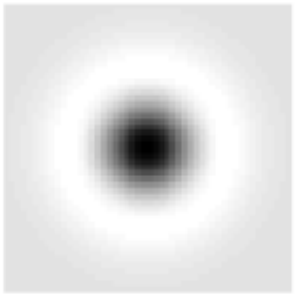
LoG



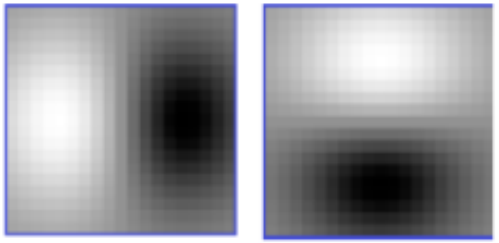
DoG

Laplacian of Gaussian : 2-D case





Compare with Derivatives of Gaussian

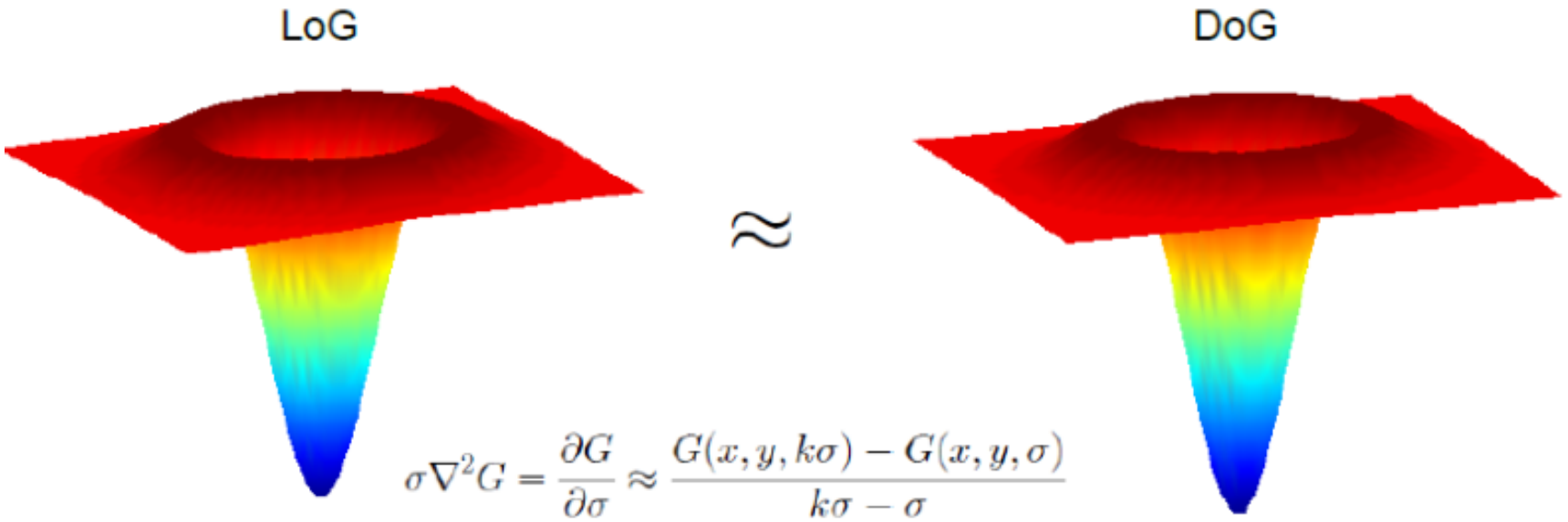


Convolution with a kernel (filter) is similar to comparing a small image of what you want to find with all local regions of the image

LoG Extrema Points

- LoG is basically a blob feature detector
- The scale (radius) of the blob is determined by sigma
- LoG extrema are stable features
- They give excellent notion of scale
- But calculation is costly

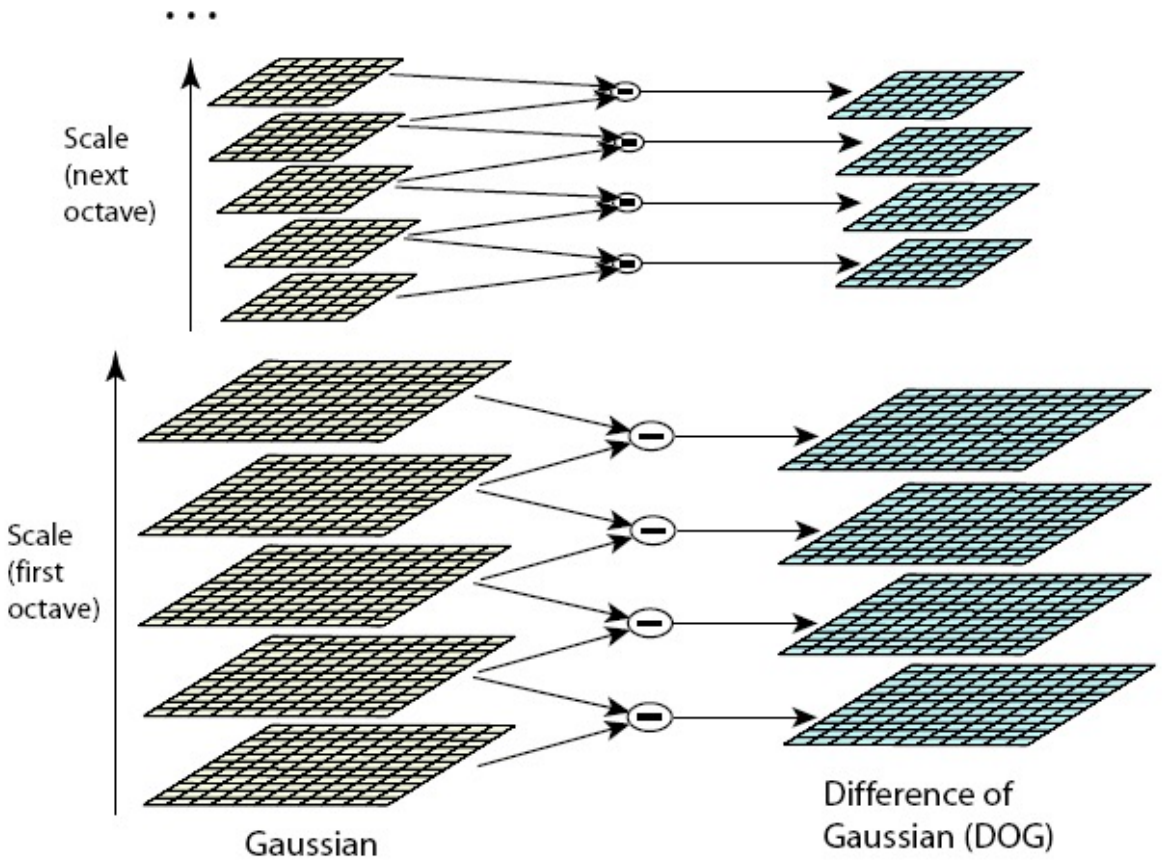
LoG approximation with DoG



$$\sigma\nabla^2 G = \frac{\partial G}{\partial \sigma} \approx \frac{G(x,y,k\sigma) - G(x,y,\sigma)}{k\sigma - \sigma}$$

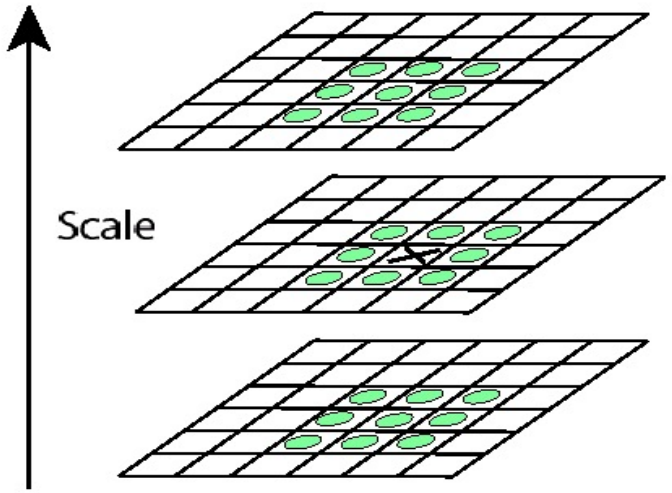
$$\begin{aligned} G(x,y,\kappa\sigma) - G(x,y,\sigma) &\approx (\kappa-1)\sigma^2\nabla^2 G \\ D(x,y,\sigma) &= (G(x,y,\kappa\sigma) - G(x,y,\sigma)) \otimes I(x,y) \\ &= L(x,y,\kappa\sigma) - L(x,y,\sigma) \end{aligned}$$

Efficient DoG Computation using Gaussian Scale Pyramid

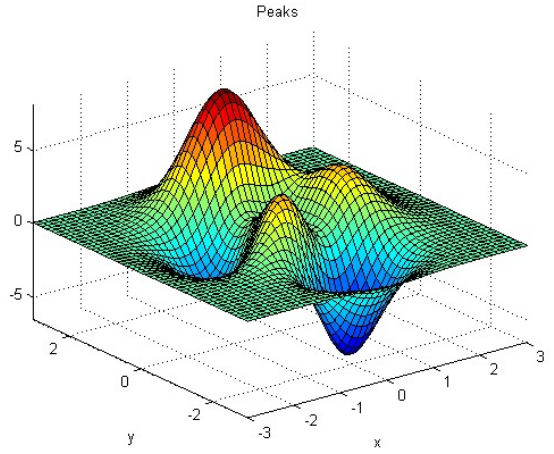


Local Extrema in DoG Images

- Minima
- Maxima
- 26 neighbourhood



Subpixel Localization of Extrema Points



- 3D surface fitting
- Recall the DoG convolved image $D(x, y, \sigma)$
- Taylor series expansion

$$D(\mathbf{x}) = D + \frac{\partial D^\top}{\partial \mathbf{x}}\mathbf{x} + \frac{1}{2}\mathbf{x}^\top \frac{\partial^2 D}{\partial \mathbf{x}^2}\mathbf{x}$$

- Where $\boldsymbol{x} = (x, y, \sigma)^\top$
- Differentiate w.r.t. $\boldsymbol{x}$ and set to zero for the extrema values of $\boldsymbol{x} = (x, y, \sigma)^\top$

$$\hat{\mathbf{x}} = \frac{\partial^2 D^{-1}}{\partial \mathbf{x}^2}\frac{\partial D}{\partial \mathbf{x}}$$

Discard Low Contrast Points

- Find the function value at extrema points $D(\hat{x})$
- Image pixel range [0 1]
- If $|D(\hat{x})|$ is less than 0.03, discard the point

Eliminate Edge Points

- Use the Hessian matrix to find edge points

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

- Derivative is low along the edge and high across i.e. only one eigenvalue is high

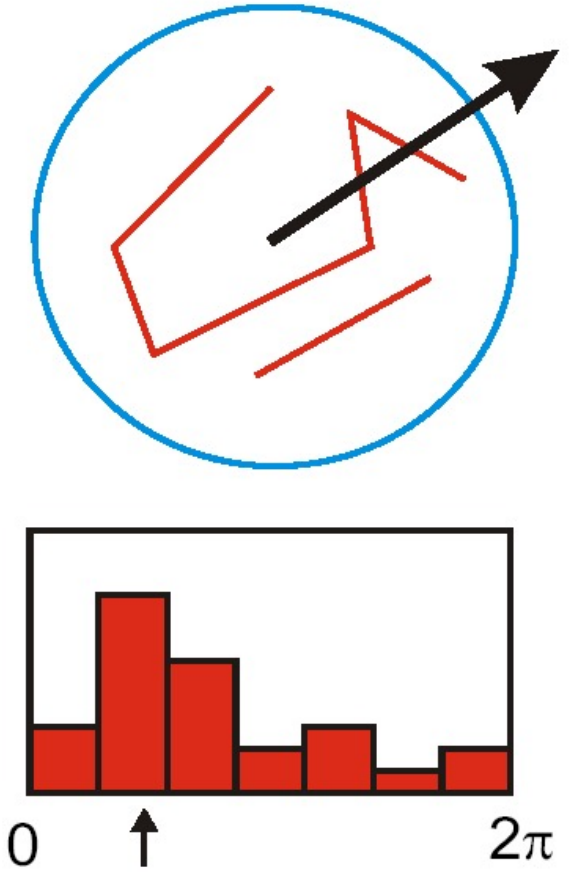
- We can avoid explicit calculation of eigenvalues since we only need their ratio $\alpha = r\beta$

$$\begin{aligned} \mathrm{Tr}(H) &= D_{xx} + D_{yy} = \alpha + \beta \\ \mathrm{Det}(H) &= D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta \\ \frac{\mathrm{Tr}(H)^2}{\mathrm{Det}(H)} &= \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r+1)^2}{r} \end{aligned}$$

This ratio should be small

Orientation Assignment : Concept

- Create histogram of local gradient directions at the selected scale
- Assign canonical orientation at the peak of the smoothed histogram
- If two major orientations, use both



Orientation Assignment : Details

- Precompute gradient magnitudes and orientations for each L

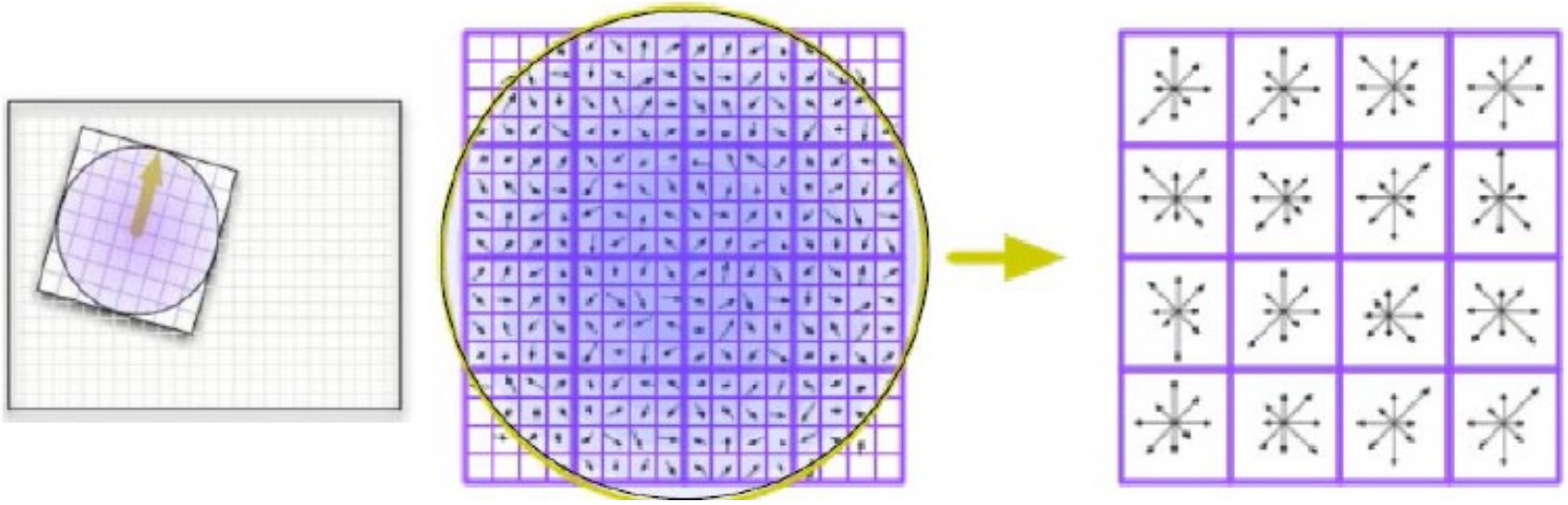
$$m(x,y) = \sqrt{(L(x+1,y)-L(x-1,y))^2+(L(x,y+1)-L(x,y-1))^2}$$

$$\theta(x,y) = \tan^{-1}\left(\frac{L(x,y+1)-L(x,y,-1)}{L(x+1,y)-L(x-1,y)}\right)$$

- Use the scale of the keypoint to select the smoothed image L
- Using a region (based on scale) around a keypoint
  - Divide orientation into 36 bins
  - Weight each point by its magnitude and
  - By a Gaussian window of 1.5σ
  - Peaks in the histogram corresponde to dominant orientations
  - Take all peaks > 0.8 of the max bin as a valid orientation
- Calculate feature for each valid orientation (usuall 1 or 2)

SIFT Feature Calculation

- Take the region around a keypoint according to its scale
- Rotate and align with the previously calculated orientation
- 8 orientation bins calculated at 4x4 bin array
- 8 x 4 x 4 = 128 dimension feature



Illumination Issues

- SIFT is a 128 dimensional vector
- For robustness to illumination, normalize the feature to unit magnitude
- To cater for image saturations, truncate the feature to 0.2
- Renormalize to unit magnitude
- SIFT properties
  - Repeatable keypoints
  - Scale invariant
  - Rotation invariant
  - Robust to viewpoint
  - Robust to illumination changes

Summary

- We studied the importance of features and properties of good features
- We learned about feature detection and feature extraction
- We studied the Harris corner detector in detail
- We studied the importance of scale and orientation invariance
- We studied how the SIFT performs keypoint detection and feature extraction in a scale and orientation invariant wa

Acknowledgements: The slides are based on Rick Szeliski's lecture notes, Computer Vision course of Penn State Univ, Open CV documentation and D. Low's IJCV paper on SIFT.